# PRO2000 - PROGRAMMING

Week 35

Gebremariam Assres
Gebremariam.assres@kristiania.no

# Today's topics

- Variables, Literals, and the Assignment Statement
- Identifiers
- Data Types: Integer, Floating-Point, char, string, bool
- Determining the Size of a Data Type
- More on Variable Assignments and Initialization
- Scope
- Arithmetic Operators
- Comments

# Variables

# Variables

- Variable
  - Has a name and a type of data it can hold




  - Is used to reference a location in memory where a value can be stored
  - Must be defined before it can be used
  - The value that is stored can be changed, *i.e.*, it can "vary"

# Variables ...

– If a new value is stored in the variable, it replaces the previous value

– The previous value is overwritten and can no longer be retrieved

```
int age;
age = 17;       // age is 17
cout << age;    // Displays 17
age = 18;       // Now age is 18
cout << age;    // Displays 18
```

# Variables ...

- Assignment Statement
  - Uses the = operator
  - Has a single variable on the left side and a value on the right side
  - Copies the value on the right into the variable on the left

```
item = 12;
```

# Variables ...

- Data item on the right can be literal
  - Whose value does not change during program execution
  - Is also called a constant

```
'A'       // character constant
"Hello"   // string literal
12        // integer constant
3.14      // floating-point constant
```

# Identifiers

- Programmer-chosen names to represent parts of the program, such as variables, constants, functions, etc.
- Name should indicate the use of the identifier
- Cannot use C++ key words as identifiers
- Must begin with alphabetic character or _, followed by alphabetic, numeric, or _ .
- Alphabetic characters may be upper- or lowercase

# Multi-word Variable Names

- Descriptive variable names may include multiple words
- Two conventions to use in naming variables:
  - Capitalize all but first letter of first word. Run words together:

    ```
    quantityOnOrder totalSales
    ```

  - Use the underscore _ character as a space:

    ```
    quantity_on_order total_sales
    ```

- Use one convention consistently throughout program

# Valid and Invalid Identifiers

| IDENTIFIER | VALID? | REASON IF INVALID |
|---|---|---|
| `totalSales` | Yes | |
| `total_Sales` | Yes | |
| `total.Sales` | No | Cannot contain period |
| `4thQtrSales` | No | Cannot begin with digit |
| `totalSale$` | No | Cannot contain $ |

# Data types

# Integer Data Types

- Designed to hold whole (non-decimal) numbers
- Can be **`signed`** or **`unsigned`**

```
12      -6      +3
```

- Available in different sizes (*i.e.*, number of bytes): **`short`**, **`int`**, and **`long`**
- Size of **`short`** ≤ size of **`int`** ≤ size of **`long`**

# Signed vs. Unsigned Integers

- C++ allocates one bit for the sign of the number. The rest of the bits are for data.
- If your program will never need negative numbers, declare as `unsigned` so all the bits are used for data.
- A variable is signed unless the `unsigned` keyword is used.

# Defining Variables

- Variables of the same type can be defined
  - In separate statements

```
int length;
int width;
```

  - In the same statement

```
int length, width;
```

- Variables of different types must be defined in separate statements

# Integral Constants

- To store an integer constant in a long memory location, put '`L`' at the end of the number: `1234L`
- Constants that begin with '`0`' (zero) are octal, or base 8: `075`
- Constants that begin with '`0x`' are hexadecimal, or base 16: `0x75A`

# Floating-Point Data Types

- Designed to hold real numbers
  `12.45` `-3.8`
- Stored in a form like scientific notation
- Numbers are all signed
- Available in different sizes (number of bytes):
  **`float`**, **`double`**, and **`long double`**
- Size of **`float`** ≤ size of **`double`**
  ≤ size of **`long double`**

# Floating-point Constants

- Are **`double`** by default and can be represented in
  - Fixed point (decimal) notation:
    `31.4159` `0.0000625`
  - E notation:
    `3.14159E1` `6.25e-5`
- Can be forced to be float `3.14159F` or long double `0.0000625L`

# Assigning Floating-point Values to Integer Variables

If a floating-point value is assigned to an integer variable

- The fractional part will be truncated (*i.e.*, “chopped off” and discarded)
- The value is not rounded

```
int rainfall = 3.88;
cout << rainfall;// Displays 3
```

# The char Data Type

- Used to hold single characters or very small integer values
- Usually occupies 1 byte of memory
- A numeric code representing the character is stored in memory

| SOURCE CODE | MEMORY | |
|---|---|---|
| `char letter = 'C';` | `letter` | 67 |

# Character Literal

- A character literal is a single character
- When referenced in a program, it is enclosed in single quotation marks:

```
cout << 'Y' << endl;
```

- The quotation marks are not part of the literal, and are not displayed

# String Literals

- Can be stored as a series of characters in consecutive memory locations

  `"Hello"`

- Stored with the null terminator, `\0`, automatically placed at the end




- Is comprised of characters between the `"  "`

# A character or a string literal?

- A character literal is a single character, enclosed in single quotes:

```
'C'
```

- A string literal is a sequence of characters enclosed in double quotes:

```
"Hello, there!"
```

- A single character in double quotes is a string literal, not a character literal:

```
"C"
```

# The C++ string Class

- Must **`#include <string>`** to create and use string objects
- Can define **`string`** variables in programs

```
string name;
```

- Can assign values to string variables with the assignment operator

```
name = "George";
```

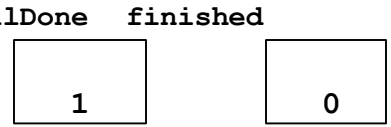
- Can display them with **`cout`**

```
cout << "My name is " << name;
```

# The bool Data Type

- Represents values that are **`true`** or **`false`**
- **`bool`** values are stored as integers
- **`false`** is represented by 0, **`true`** by 1

```
bool allDone = true;
bool finished = false;
```



# Determining the Size of a Data Type

The **`sizeof`** operator gives the size in number of bytes of any data type or variable

```
double amount;

cout  <<  "A float is stored in "
      <<  sizeof(float) << " bytes\n";
cout  <<  "Variable amount is stored in          "
      <<  sizeof(amount)<< " bytes\n";
```

# More on Variable Assignments and Initialization

Assigning a value to a variable

- Assigns a value to a previously created variable
- A single variable name must appear on left side of the = symbol

```
int size;
size = 5;   // legal
5 = size;   // not legal
```

# Variable Assignment vs. Initialization

Initializing a variable

- Gives an initial value to a variable at the time it is created
- Can initialize some or all of the variables being defined

```
int length = 12;
int width = 7, height = 5, area;
```

# Scope

- The scope of a variable is that part of the program where the variable may be used
- A variable cannot be used before it is defined

```
int num1 = 5;
cout >> num1;      // legal
cout >> num2;      // illegal
int num2 = 12;
```

# Operators

# Arithmetic Operators

- Used for performing numeric calculations
- C++ has unary, binary, and ternary operators
  - unary (1 operand) `-5`
  - binary (2 operands) `13 - 7`
  - ternary (3 operands) `exp1 ? exp2 : exp3`

# Binary Arithmetic Operators

| SYMBOL | OPERATION | EXAMPLE | ans |
|---|---|---|---|
| + | addition | `ans = 7 + 3;` | 10 |
| - | subtraction | `ans = 7 - 3;` | 4 |
| * | multiplication | `ans = 7 * 3;` | 21 |
| / | division | `ans = 7 / 3;` | 2 |
| % | modulus | `ans = 7 % 3;` | 1 |

# / Operator

- C++ division operator (`/`) performs integer division if both operands are integers

```
cout << 13 / 5;    // displays 2
cout <<  2 / 4;    // displays 0
```

- If either operand is floating-point, the result is floating-point

```
cout << 13 / 5.0;  // displays 2.6
cout << 2.0 / 4;   // displays 0.5
cout << 2.0 / 0;   // illegal
```

# % Operator

- C++ modulus operator (`%`) computes the remainder resulting from integer division

```
cout << 9 % 2;   // displays 1
```

- `%` requires integers for both operands

```
cout << 9 % 2.0; // error
```

# Arithmetic

```
#include <iostream>
int main() {
   int a = 10, b = 3;
   std::cout << "a + b = " << a + b << "\n"; // prints 13
   std::cout << "a / b = " << a / b << "\n"; // prints 3 (integer division)
   std::cout << "a % b = " << a % b << "\n"; // prints 1
   a += 5; // a = 15

   int i = 5;
   std::cout << i++ << "\n"; // prints 5, then i becomes 6 (post-increment)
   std::cout << ++i << "\n"; // increments to 7, prints 7 (pre-increment)
}
```

# Arithmetic precedence and relational

```
#include <iostream>

int main() {

        // * has higher precedence than +
        int a = 2 + 3 * 4;    // 3*4 evaluated first -> 2 + 12 = 14
        int b = (2 + 3) * 4; // parentheses change grouping -> 5 * 4 = 20

        std::cout << "a = 2 + 3 * 4 -> " << a << '\n';
        std::cout << "b = (2 + 3) * 4 -> " << b << '\n';

        // Logical/ relational
        int a = 5, b = 10;
        bool cmp = (a < b) && (b > 0);
        std::cout << std::boolalpha << cmp << "\n"; // true

        return 0;
}
```

# Associativity of division (left-to-right)

```
#include <iostream>

int main() {

        // / is left-associative: (100 / 10) / 2
        int divLeft  = 100 / 10 / 2;
        // Parentheses change it: 100 / (10 / 2)
        int divRight = 100 / (10 / 2);

        std::cout << "divLeft  = 100 / 10 / 2 -> " << divLeft << '\n';
        std::cout << "divRight = 100 / (10 / 2) -> " << divRight << '\n';

        return 0;
}
```

Division groups left-to-right by default.
To change grouping, use parentheses.

**Expected output:**

divLeft  = 100 / 10 / 2 -> 5
divRight = 100 / (10 / 2) -> 20

# Additive (+) vs left-shift (<<) precedence

```
#include <iostream>

int main() {

        // + has higher precedence than <<, so 2 + 1 is done before shift
        int shift1 = 1 << 2 + 1;      // parsed as 1 << (2 + 1) -> 1 << 3 = 8
        int shift2 = (1 << 2) + 1;    // parsed as (1 << 2) + 1 -> 4 + 1 = 5

        std::cout << "shift1 = 1 << 2 + 1 -> " << shift1 << '\n';
        std::cout << "shift2 = (1 << 2) + 1 -> " << shift2 << '\n';
        return 0;

}
```

Addition is evaluated before the bit-shift operator unless parentheses change the order.

**Expected output**

shift1 = 1 << 2 + 1 -> 8
shift2 = (1 << 2) + 1 -> 5

# More on C++ features

- C++ is case-sensitive
  - Uppercase and lowercase characters are different characters. Example, ‘Main’ is not the same as ‘main’.
- Comments- used to document parts of a program
  - Indicate the purpose of the program; ddescribe the use of variables; explain complex sections of code
  - Single line commets- // comment
  - Multi-Line Comments- Begin with `/*` and end with `*/`

# More on C++ features ...

| Escape Sequence | Name | Description |
|---|---|---|
| `\n` | Newline | Causes the cursor to go to the next line for subsequent printing. |
| `\t` | Horizontal tab | Causes the cursor to skip over to the next tab stop. |
| `\a` | Alarm | Causes the computer to beep. |
| `\b` | Backspace | Causes the cursor to back up, or move left one position. |
| `\r` | Return | Causes the cursor to go to the beginning of the current line, not the next line. |
| `\\` | Backslash | Causes a backslash to be printed. |
| `\'` | Single quote | Causes a single quotation mark to be printed. |
| `\"` | Double quote | Causes a double quotation mark to be printed. |

# Up next: Exercises

- Now: 2 hours (12:15 to 14:00) of exercises (found on Canvas).

- Short exercise summary:
  - *Write, compile, and debug a C++ programs with emphasis on variables, data types, and operators*
- Remember:
  - You should do approximately 4 more hours of the course (in a study group or alone) *before* our next session together. :-)